ACCENTURE 2026–2027 HIRING · TECHNICAL ASSESSMENT · SECTION D

Computer Science Fundamentals Complete Guide

> ≈ 14 of 45 questions · concepts + small logic · programming · DSA · DBMS_

Everything for Section D, from zero: how programs run, data types and memory, functions and pointers, arrays and strings, every core data structure, searching and sorting with step-by-step passes, Big-O, and DBMS with SQL. Includes 56 practice MCQs, every answer produced by running code, computing the data structure, or querying a real database.

Programming basics | Memory & pointers | Data structures | Search & sort | Big-O | DBMS & SQL

≈ 14	6	56	gcc · JDK · SQLite
Questions in the test	Sorting algorithms traced	Practice MCQs	Answers verified by execution

Independent preparation guide · Not affiliated with or endorsed by Accenture

Edition: 2026–2027

INSIDE THIS GUIDE

Contents

00 What's tested and how to prepare

F1 How programs run: compilers and interpreters

F2 Data types, ranges and casting

F3 Variables, scope and control flow

F4 Functions, memory and pointers

F5 Arrays and strings

F6 Linear data structures

F7 Trees, heaps, graphs and hashing

F8 Searching and sorting

F9 Time and space complexity

DB1 DBMS concepts and keys

DB2 ER model and normalisation

DB3 SQL: commands, clauses and joins

DB4 Transactions, ACID and concurrency

✓ Practice sets, answer keys and cheat sheet

KNOW THE SECTION

00 What's Tested & How to Prepare

About **14 of the 45** questions. They mix **definitions** (which structure is LIFO? what is 3NF?), **small logic** (stack sequences, a sorting pass, a traversal, a loop's complexity) and **short code or SQL outputs**.

Area	Typical questions	Sections
Programming basics	Compiler vs interpreter · data type sizes and ranges · casting · scope · call by value vs reference · pointers	F1–F5
Data structures	LIFO/FIFO · push/pop sequences · postfix · linked-list complexity · tree traversals · heaps · hashing	F6–F7
Algorithms	Binary search conditions · array after one sorting pass · best/worst cases · stability · Big-O of loops	F8–F9
DBMS	Keys · normal forms · DDL/DML · DELETE vs TRUNCATE · joins · GROUP BY/HAVING · ACID	DB1–DB4

How to prepare

Learn the **comparison tables** (sorting, data structures, keys and normal forms) and practise the **hands-on skills**: tracing a stack, doing one pass of each sort, traversing a small tree, and running a GROUP BY by hand on the sample tables.

PART 1 · PROGRAMMING FOUNDATIONS

Programming Fundamentals

F

How code becomes a running program, how data is stored, and how functions, memory, arrays and strings work. These are the basics behind every other topic.

F1

FROM SOURCE TO EXECUTION

How Programs Run

`Source code (.c)` → **preprocessor** → `Compiler` → `Object code (.o)` → **linker** → `Executable` → **loader** → `Running in memory`

Tool	What it does
Compiler	Translates the **whole program** into machine code before running it. Errors are reported together, and the program runs fast (C, C++, Go)
Interpreter	Translates and runs **line by line**. It stops at the first error and runs more slowly (Python, JavaScript, Ruby)
Hybrid (Java, C#)	Compiled to **bytecode**, which runs on a virtual machine (the **JVM**/CLR). A **JIT** compiler turns hot code into native code. "Write once, run anywhere"
Assembler	Assembly language → machine code
Linker / Loader	The linker combines object files and libraries into an executable. The loader places the executable in memory to run

Language levels
Machine language (binary) → **assembly** (mnemonics like MOV, ADD) → **high-level** (C, Java, Python). C is often called "middle level".

Paradigms
Procedural (C) · **Object-oriented** (Java, C++) · **Functional** (Haskell, plus lambdas in many languages) · **Scripting** (Python, JS).

Error types
Syntax (compile time: a missing ;) · **Runtime** (divide by zero, null pointer) · **Logical** (runs, but gives the wrong answer).

F2

HOW DATA IS STORED

Data Types, Ranges & Casting

Type	C (typical, 64-bit)	Java (fixed)
char	1 byte (ASCII)	2 bytes (Unicode)
short	2 bytes	2 bytes
int	4 bytes*	4 bytes
long	8 bytes (4 on Windows)	8 bytes
float / double	4 / 8 bytes	4 / 8 bytes

Integer ranges
n bits, **signed**: -2^{n-1} to $2^{n-1} - 1$ · **unsigned**: 0 to $2^n - 1$.
8-bit: −128...127 / 0...255 · 16-bit: −32,768...32,767 · 32-bit: about ±2.147 billion.

Overflow wraps around
unsigned char 255 + 1 = **0** · Java's `Integer.MAX_VALUE + 1` = `Integer.MIN_VALUE`.

Type	C (typical, 64-bit)	Java (fixed)
byte · boolean	— · `_Bool` (1 byte)	1 byte · true/false

*C only guarantees minimum sizes (int ≥ 16 bits). Java's sizes are fixed on every platform.

Implicit (widening) conversion: done automatically when no data is lost. byte → short → int → long → float → double. In mixed expressions, the smaller type is promoted: `5 / 2.0` = 2.5, but `5 / 2` = 2.

Explicit (narrowing) casting: you have to ask for it, and data may be lost. `(int) 7.9` = 7 (truncates toward 0) · `(int) -7.9` = −7 · `(char) 66` = 'B' · `(byte) 130` = −126.

Floating point: stored in binary (IEEE 754), so many decimals are approximations. `0.1 + 0.2 == 0.3` is **false**. float has about 7 significant digits and double about 15–16. Never compare floats with ==; check whether they're close enough instead.

F3

WHERE VARIABLES LIVE

Variables, Scope & Control Flow

Kind	Scope (where it's visible)	Lifetime (how long it exists)
Local	Inside its block or function	Until the block ends
Global	The whole file or program	The whole program run
static local (C)	Its function only	The whole program: initialised **once** and keeps its value between calls
Instance field	Through the object	As long as the object exists
static field	Through the class	As long as the class is loaded, with one shared copy

An inner variable with the same name **shadows** the outer one inside its block. **Constants**: `const` (C) and `final` (Java). Control flow: **sequence** → **selection** (if, switch) → **iteration** (for, while, do-while). **Recursion vs iteration**: recursion is cleaner for trees and divide-and-conquer, but uses stack memory. Iteration is usually faster and uses O(1) space.

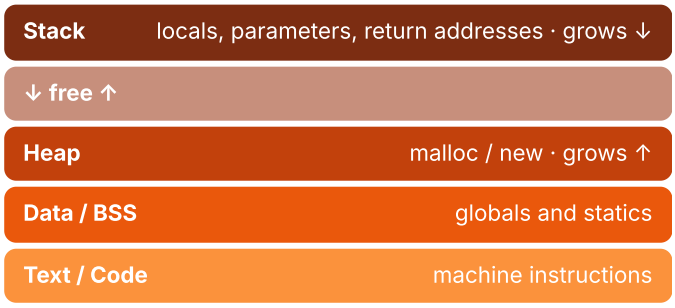
F4

FUNCTIONS & MEMORY

Functions, Memory & Pointers

Parameters vs arguments
Parameters are the variables in the function definition. Arguments are the values passed in the call. A **declaration** (prototype) gives the signature, and a **definition** provides the body.

Memory layout of a running C program



Each function call pushes a **stack frame**. Recursion that's too deep causes a **stack overflow**.

Call by value vs reference
By value: a copy is passed, so the caller is unchanged (C's default, and Java's primitives). **By reference / pointer**: the function works on the original (C pointers, C++ `&`). Java passes object **references by value**, so it can change an object's fields but can't reassign the caller's variable.

Pointer concept (C)	Meaning	Example
Pointer	A variable that stores an **address**	`int *p = &x;`
& (address-of)	Gives a variable's address	`&x`
*** (dereference)**	The value at that address	`*p = 10;` changes x
Pointer arithmetic	`p + 1` moves by **one element** (sizeof(type) bytes)	`*(a + 2) == a[2]`
NULL pointer	Points to nothing. Dereferencing it crashes the program	`int *p = NULL;`
Dangling pointer	Points to memory that has been freed	use after `free(p)`
Memory leak	Heap memory that's never freed	`malloc` without `free`
Dynamic allocation	Memory requested at run time, on the heap	`malloc / calloc / realloc / free` · C++ `new / delete`

Java has no explicit pointers. It uses references, and the **garbage collector** frees unreachable objects automatically. **Inline functions** (C/C++) ask the compiler to paste the body in place of the call. **Function overloading** means the same name with different parameters (C++ and Java, but not C).

F5

COLLECTIONS OF DATA

Arrays & Strings

Array: a fixed-size, **contiguous** block of same-type elements, 0-indexed, with O(1) random access.
1D address = base + i × size.
2D row-major (C, Java): base + (i × columns + j) × size.
Column-major (Fortran, MATLAB): base + (j × rows + i) × size.

Strings
C: a char array ending in `'\0'`. `strlen` doesn't count it, but `sizeof` does. Functions: strlen, strcpy, strcat, strcmp (returns 0 when equal), strrev.
Java: `String` is **immutable**, so methods return new strings. Use `StringBuilder` (fast, mutable) or `StringBuffer` (thread-safe) for repeated changes.

Java String method	Example → result	Java String method	Example → result
`length()`	"code".length() → 4	`indexOf(s)`	"banana".indexOf("an") → 1
`charAt(i)`	"code".charAt(1) → 'o'	`equals / equalsIgnoreCase`	content comparison
`substring(a, b)`	"program".substring(2, 5) → "ogr"	`compareTo`	"a".compareTo("c") → −2
`toUpperCase()`	"ab".toUpperCase() → "AB"	`split(",")`	"a,b".split(",") → [a, b]

Java String method	Example → result	Java String method	Example → result
`trim()`	" hi ".trim() → "hi"	`replace(a, b)`	"aab".replace('a', 'x') → "xxb"

substring(a, b) includes index a and **excludes** index b. Concatenation is evaluated left to right: `1 + 2 + "x"` → "3x", but `"x" + 1 + 2` → "x12".

F6

ORGANISING DATA

Linear Data Structures

Classification: **linear** (elements in a sequence: array, linked list, stack, queue) vs **non-linear** (hierarchical or networked: tree, graph) · **static** (fixed size: array) vs **dynamic** (grows: linked list) · a **primitive** type (int, char) vs an **abstract data type** (ADT: stack, queue, defined by its operations).

	Array	Singly linked list
Memory	Contiguous, fixed size	Scattered nodes (data + next pointer), dynamic
Access the i-th element	**O(1)**	O(n): you must traverse
Insert or delete at the front	O(n): shift everything	**O(1)**
Insert at the end	O(1) if there's space	O(n), or O(1) with a tail pointer
Extra memory	None	A pointer per node
Cache friendliness	High	Low

Linked-list types: **singly** (next) · **doubly** (prev + next, so it can go backwards, and deleting a known node is O(1)) · **circular** (the last node points to the head). To **reverse** a singly list, use three pointers: prev, curr and next.

Stack: LIFO (Last In, First Out)

Operations: `push`, `pop`, `peek/top`, `isEmpty`, all O(1). **Overflow** = pushing onto a full stack. **Underflow** = popping an empty one.

Uses: the function call stack and recursion, undo/redo, the browser back button, balanced brackets, expression conversion and evaluation, DFS, backtracking.

`push 10` `push 20` `pop → 20` `push 30` `push 40` `pop → 40` `pop → 30`

Queue: FIFO (First In, First Out)

`enqueue` at the **rear**, `dequeue` from the **front**, both O(1).

Circular queue: rear = (rear + 1) mod n, which reuses the space freed at the front. **Deque**: insert and delete at both ends. **Priority queue**: highest priority out first (usually a heap).

Uses: CPU and printer scheduling, BFS, buffers, call centres.

Infix → postfix (shunting-yard)

1. Operand → write it to the output.
2. `(` → push it. `)` → pop to the output until you reach `(`.
3. Operator → pop operators of **higher or equal** precedence to the output, then push it.
4. At the end, pop everything left.

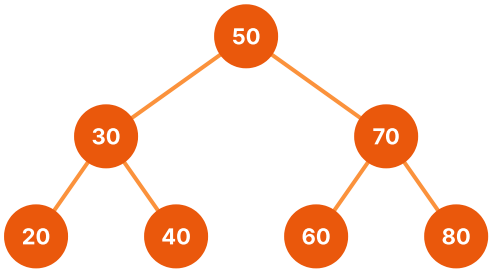
`A + B * C - D` → **A B C * + D -** · `(A + B) * C` → **A B + C ***

Evaluating postfix with a stack Scan left to right. Push numbers. When you meet an operator, pop the **top two** (second-popped OP first-popped) and push the result.
`5 1 2 + 4 * + 3 -`: 1 + 2 = 3 → 3 × 4 = 12 → 5 + 12 = 17 → 17 − 3 = **14**.
Prefix (Polish notation) works the same way, scanning right to left.

F7

HIERARCHIES & NETWORKS

Trees, Heaps, Graphs & Hashing



Traversal	Order	Result for this BST
In-order	L, Root, R	20 30 40 50 60 70 80 (**sorted**)
Pre-order	Root, L, R	50 30 20 40 70 60 80
Post-order	L, R, Root	20 40 30 60 80 70 50
Level-order	BFS (queue)	50 30 70 20 40 60 80

Tree terms: root · parent/child · siblings · **leaf** (no children) · internal node · **depth** (edges from the root) · **height** (longest path down to a leaf) · **degree** (number of children) · subtree.

Binary tree facts (root at level 0)
Max nodes at level i = 2^i · max nodes at height h = $2^{h+1} - 1$ · min height for n nodes ≈ $\lfloor \log_2 n \rfloor$ · a tree with n nodes has **n − 1 edges** · leaves = nodes with 2 children + 1 (in a full binary tree).

Types: **full** (0 or 2 children each) · **complete** (every level full except possibly the last, filled left to right: heaps) · **perfect** · **skewed** (a chain).

BST
left < node < right. Search, insert and delete are **O(log n)** on average but **O(n)** if skewed (sorted input). **AVL** and **Red-Black** trees self-balance to guarantee O(log n).

Heap (complete binary tree in an array)
Min-heap: parent ≤ children (min at the root). **Max-heap**: parent ≥ children. For index i, the children are **2i + 1, 2i + 2** and the parent is **(i − 1)/2**. Insert and delete are O(log n), peek is O(1), and building a heap is O(n).

Graphs
Vertices + edges · directed or undirected · weighted · cyclic or acyclic (a DAG). Stored as an **adjacency matrix** (V^2 space, O(1) edge check) or an **adjacency list** (V + E space, better for sparse graphs). **BFS** uses a queue (shortest path in unweighted graphs). **DFS** uses a stack or recursion.

Hashing: a hash function maps a key to an index, giving O(1) average search, insert and delete. A **collision** happens when two keys map to the same index. **Load factor** α = n / table size, and you rehash when it gets high.

Collision resolution: **chaining** (a linked list at each index) · **open addressing**: **linear probing** (i + 1, i + 2..., which causes clustering), quadratic probing (i + 1^2, i + 2^2...), double hashing.

Also good to recognise: a **trie** (prefix tree, for autocomplete) · **B / B+ trees** (database indexes) · **graph algorithms**: Dijkstra (shortest path), Prim and Kruskal (minimum spanning tree), topological sort (DAGs).

F8

FIND AND ORDER

Searching & Sorting

Linear search: check each element. O(n), and works on **any** array. Best case O(1) (the first element).

Binary search: needs **sorted** data with random access. Compare with the `mid = (lo + hi) / 2` element and discard half each step. O(log n), so at most $\lfloor \log_2 n \rfloor + 1$ comparisons (1,000 elements → 10). It doesn't work efficiently on linked lists.

One pass of each sort on `[5, 1, 4, 2, 8]`

Algorithm	What one pass does	After pass 1
Bubble	Swap adjacent pairs that are out of order. The **largest** value bubbles to the end	[1, 4, 2, 5, **8**]
Selection	Find the **minimum** of the unsorted part and swap it to the front	[**1**, 5, 4, 2, 8]
Insertion	Take the next element and insert it into the sorted left part	[**1, 5**, 4, 2, 8]
Merge	Split into halves recursively, then **merge** the sorted halves	[1, 5] [4, 2, 8] → ... → sorted
Quick	Pick a **pivot**, then partition: smaller values left, larger values right. Recurse on each side	pivot 8 → [5, 1, 4, 2] **8**
Heap	Build a max-heap, then repeatedly swap the root to the end and re-heapify	max (8) moves to the end

Algorithm	Best	Average	Worst	Space	Stable	In-place
Bubble	O(n)*	$O(n^2)$	$O(n^2)$	O(1)	✓	✓
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	×	✓
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	✓	✓
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	✓	×
Quick	O(n log n)	O(n log n)	**$O(n^2)$**	O(log n)	×	✓
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	×	✓
Counting / Radix	O(n + k)	O(n + k)	O(n + k)	O(n + k)	✓	×

*Bubble sort's best case assumes the "stop if no swaps" check. A **stable** sort keeps equal elements in their original order. **In-place** means it needs only O(1) extra space (quick sort uses O(log n) for recursion). Comparison sorts can't beat O(n log n) in

the worst case.

Which sort when? Nearly sorted or small data → **insertion** · guaranteed n log n or linked lists → **merge** · fastest in practice for in-memory arrays → **quick** · O(1) space with guaranteed n log n → **heap** · fewest swaps → **selection** (n − 1 swaps) · small-range integers → **counting**.

F9

HOW FAST, HOW BIG

Time & Space Complexity

Big-O (O)
An **upper bound**, used for the worst case. "Grows no faster than..."

Big-Omega (Ω)
A **lower bound**, linked to the best case. "Grows at least as fast as..."

Big-Theta (Θ)
A **tight bound**: both O and Ω. "Grows exactly like..."

Rules: drop constants (O(3n) = O(n)) · keep the dominant term (O(n^2 + n log n) = O(n^2)) · sequential steps **add** and nested steps **multiply** · different inputs keep separate variables (O(a + b) or O(a·b)). **Growth order**: O(1) < O(log n) < O(√n) < O(n) < O(n log n) < O(n^2) < O(n^3) < O(2^n) < O(n!).

Code pattern	Complexity
Fixed steps, or an array access by index	O(1)
`for (i = 1; i < n; i *= 2)` or `i /= 2` · binary search	O(log n)
`for (i = 1; i * i <= n; i++)` · checking if a number is prime	O(√n)
`for (i = 0; i < n; i++)` · linear search	O(n)
Two separate (non-nested) loops to n	O(n)
`for i to n` containing `for (j = 1; j < n; j *= 2)` · merge sort · heap sort	O(n log n)
Nested loops to n, including `j = i to n` (n(n+1)/2 steps)	O(n^2)
Three nested loops	O(n^3)
Naive recursive Fibonacci · generating all subsets	O(2^n)
Generating all permutations	O(n!)

Recurrence	Solves to	Example
T(n) = T(n − 1) + O(1)	O(n)	Factorial, linear recursion
T(n) = T(n/2) + O(1)	O(log n)	Binary search
T(n) = 2T(n/2) + O(n)	O(n log n)	Merge sort, average quick sort
T(n) = T(n − 1) + O(n)	O(n^2)	Worst-case quick sort, selection sort
T(n) = 2T(n − 1) + O(1)	O(2^n)	Tower of Hanoi (2^n − 1 moves), naive Fibonacci

Space complexity is the extra memory used: iterative sum O(1) · copying an array O(n) · recursion depth d uses O(d) stack space (recursive factorial O(n), binary search recursion O(log n)) · merge sort O(n).

PART 2 · DATABASE MANAGEMENT SYSTEMS

DBMS & SQL

DB

How data is modelled, organised and queried. Expect definitions (keys, normal forms, ACID), command families, and simple query outputs on small tables.

DB1

FOUNDATIONS

DBMS Concepts & Keys

Why a DBMS instead of files?
It reduces **redundancy and inconsistency**, and provides concurrent access, security, integrity constraints, backup and recovery, and a query language (SQL). An **RDBMS** stores data in related **tables**: MySQL, Oracle, PostgreSQL, SQL Server. **NoSQL** databases (MongoDB, Cassandra, Redis) use documents, columns or key-value pairs.

Term	Meaning
Relation	A table
Tuple	A row (record)
Attribute	A column (field)
Degree	Number of **columns**
Cardinality	Number of **rows**
Domain	The allowed values of an attribute
Schema / Instance	The structure / the data at a given moment

Key	Definition	Example (Student table)
Super key	Any set of columns that uniquely identifies a row	{roll}, {roll, name}, {email}
Candidate key	A **minimal** super key (no unnecessary columns)	{roll}, {email}
Primary key	The chosen candidate key: **unique + NOT NULL**, one per table	roll
Alternate key	The candidate keys not chosen as primary	email
Foreign key	A column referencing another table's primary key (**referential integrity**)	dept_id → Department
Composite key	A key made of 2 or more columns	{student_id, course_id}
Unique key	Values must be unique, but NULL is allowed	phone

Constraints: NOT NULL · UNIQUE · PRIMARY KEY · FOREIGN KEY (ON DELETE CASCADE / SET NULL) · CHECK (`salary > 0`) · DEFAULT. **Integrity types**: entity (the primary key isn't null), referential (a foreign key matches an existing primary key), domain (valid values).

DESIGN

DB2 ER Model & Normalisation

ER diagram symbols
Entity = rectangle (a weak entity has a double rectangle) · **attribute** = oval (key = underlined, multi-valued = double oval, derived = dashed) · **relationship** = diamond · cardinality **1:1, 1:N, M:N** · participation total (double line) or partial. An M:N relationship becomes a **separate table**.

Why normalise? To remove redundancy and the **anomalies** it causes: **insertion** (you can't add data without unrelated data) · **update** (the same fact stored in many rows gets out of sync) · **deletion** (deleting one fact loses another).

Form	Rule	Fix
1NF	Atomic values: no lists or repeating groups in a cell	"Java, SQL" in one cell → one row per skill
2NF	1NF + **no partial dependency** (a non-key column depending on *part* of a composite key)	(student, course) → student_name? Move student_name to a Student table
3NF	2NF + **no transitive dependency** (non-key → non-key)	emp → dept_id → dept_name? Move dept_name to a Department table
BCNF	For every dependency X → Y, X is a super key (a stricter 3NF)	Decompose further

Higher forms (4NF for multi-valued dependencies, 5NF for join dependencies) are rarely asked. **Denormalisation** deliberately adds redundancy for read speed, for example in data warehouses.

THE QUERY LANGUAGE

DB3 SQL: Commands, Clauses & Joins

Family	Commands	Note
DDL (Definition)	CREATE, ALTER, DROP, TRUNCATE, RENAME	Changes the structure. Auto-commits
DML (Manipulation)	INSERT, UPDATE, DELETE (MERGE)	Changes the data. Can be rolled back
DQL (Query)	SELECT	Reads data
DCL (Control)	GRANT, REVOKE	Permissions
TCL (Transaction)	COMMIT, ROLLBACK, SAVEPOINT	Transactions

DELETE
DML. Removes rows (supports WHERE). Can be rolled back, and triggers fire. Slower.

TRUNCATE
DDL. Removes **all** rows quickly, keeps the structure, and resets identity counters. No WHERE.

DROP
DDL. Removes the **whole table**: its structure, data and indexes.

Clause order: written vs executed
Written: SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT
Executed: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT
That's why WHERE can't use aggregates or SELECT aliases, but ORDER BY can.

Aggregates & NULL
COUNT, SUM, AVG, MIN and MAX **ignore NULLs**, except `COUNT(*)`, which counts rows. `NULL = NULL` isn't true, so use `IS NULL`. Any arithmetic with NULL gives NULL. **GROUP BY puts all NULLs into one group.** Use `COALESCE(x, 0)` to replace NULLs.

Sample tables (used in the SQL practice questions)

Employee

id	name	dept_id	salary	manager_id
1	Asha	10	40000	NULL
2	Ravi	20	55000	1
3	Meena	20	62000	1
4	John	30	38000	2
5	Priya	10	45000	2
6	Arjun	NULL	51000	3

Department

dept_id	dname
10	HR
20	IT
30	Sales
40	Finance

Arjun has no department, and Finance has no employees. Both are deliberate, so you can see how joins treat unmatched rows.

Join	Returns	On the sample tables
INNER JOIN	Only rows that match in **both** tables	5 rows (no Arjun, no Finance)
LEFT JOIN	All rows of the left table, with NULLs where there's no match	Employee LEFT JOIN Dept: 6 rows
RIGHT JOIN	All rows of the right table	Employee RIGHT JOIN Dept: 6 rows (with Finance)
FULL OUTER	All rows of both tables	7 rows (Arjun and Finance)
CROSS JOIN	The Cartesian product: every pair	6 × 4 = 24 rows
SELF JOIN	A table joined with itself	Employee ↔ their manager

Subqueries
A query inside another. Second-highest salary: `SELECT MAX(salary) FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee)`. Operators: IN, EXISTS, ANY, ALL.

UNION vs UNION ALL
UNION removes duplicates. UNION ALL keeps them (and is faster). The columns must match in number and type. INTERSECT and EXCEPT/MINUS also exist.

View · Index
A **view** is a saved query, used like a virtual table. An **index** speeds up reads (usually a B+ tree) but slows writes. Clustered = sorts the table itself (one per table).

Handy operators: `BETWEEN 10 AND 20` (inclusive) · `IN (…)` · `LIKE 'A%'` (% = any characters, _ = exactly one) · `DISTINCT` · `ORDER BY salary DESC` · `LIMIT n` / `TOP n`. A **stored procedure** is saved SQL code. A **trigger** runs

automatically on INSERT, UPDATE or DELETE.

DB4

RELIABILITY

Transactions, ACID & Concurrency

Atomicity
All or nothing. A transfer debits **and** credits, or neither happens.

Consistency
Moves from one valid state to another, and constraints hold.

Isolation
Concurrent transactions don't see each other's partial work.

Durability
Committed data survives crashes, using logs and disk writes.

Problem	What happens	Prevented from
Dirty read	Reads data that another transaction hasn't committed yet (and may roll back)	Read Committed
Non-repeatable read	The same row read twice gives different values	Repeatable Read
Phantom read	The same query returns new or missing rows	Serializable
Lost update	Two writes overlap and one is overwritten	Locking
Deadlock	Two transactions each wait for the other's lock	Detection or timeouts, lock ordering

Isolation levels, from weakest to strongest: Read Uncommitted → Read Committed → Repeatable Read → **Serializable**. Transaction states: Active → Partially Committed → Committed / Failed → Aborted. **Locks**: shared (read) and exclusive (write).

CODE OUTPUT

Practice Set 1 · Programming

P1–P10 are C programs, **compiled and run with gcc**. P11–P14 are Java programs, **run with the JDK**. The answer key is the real program output.

P1 Pointer arithmetic — What is the output? (C)

```
int a[] = {10, 20, 30, 40};
int *p = a;
p += 2;
printf("%d", *p + 1);
```

A 31
B 12
C 41
D 21

P2 Dereferencing — What is the output? (C)

```
int x = 5;
int *p = &x;
*p = *p * 2;
printf("%d", x);
```

A 25
B 10
C 5
D Address of x

P3 Swap via pointers — What is the output? (C)

```
void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }
int x = 3, y = 7;
swap(&x, &y);
printf("%d %d", x, y);
```

A 3 7
B 7 3
C 7 7
D 3 3

P4 static local variable — What is the output? (C)

```
void counter() { static int c = 0; c++; printf("%d ", c); }
counter(); counter(); counter();
```

A 3 3 3
B 0 1 2
C 1 2 3
D 1 1 1

P5 strlen and \0 — What is the output? (C)

```
printf("%d", (int) strlen("hello\0world"));
```

A 5
B 11
C 12
D 10

P6 sizeof a char array — What is the output? (C)

```
char s[] = "abc";
printf("%d", (int) sizeof(s));
```

A 3
B 8
C 1
D 4

P7 unsigned overflow — What is the output? (C)

```
unsigned char c = 255;
c++;
printf("%d", c);
```

A 255
B 0
C -1
D 256

P8 Casting truncates — What is the output? (C)

```
printf("%d", (int) 7.9 + (int) -7.9);
```

A 0
B -1
C 1
D 16

P9 Block scope — What is the output? (C)

```
int i = 10;
{
    int i = 20;
    i++;
}
printf("%d", i);
```

A 20
B 21
C 10
D 11

P10 Recursive array sum — What is the output? (C)

```
int sum(int a[], int n) { return n == 0 ? 0 : a[n-1] + sum(a, n-1); }
int a[] = {4, 7, 1, 9};
printf("%d", sum(a, 4));
```

A 9
B 21
C 13
D 20

P11 String concatenation order — What is the output? (Java)

```
System.out.print(10 + 20 + "x" + 10 + 20);
```

A 1020×30
B 1020×1020
C 30×1020
D 30×30

P12 Strings are immutable — What is the output? (Java)

```
String s = "abc";
s.concat("d");
System.out.print(s);
```

A abcd
B d
C null
D abc

P13 StringBuilder is mutable — What is the output? (Java)

```
StringBuilder sb = new StringBuilder("abc");
sb.append("d").reverse();
System.out.print(sb);
```

A abc
B abcd
C dcba
D cbad

P14 char arithmetic — What is the output? (Java)

```
char c = 'a';
System.out.print((c + 1) + " " + (char)(c + 1));
```

A 98 98
B 98 b
C b b
D a1 b

Set 1 · Answer key

Q	Ans	Correct	Why
P1	A	`31`	p += 2 moves two elements ahead, to a[2] = 30. *p + 1 = 31.
P2	B	`10`	*p IS x, so changing *p changes x.
P3	B	`7 3`	Passing addresses (by reference) lets the function change the caller's variables.
P4	C	`1 2 3`	A static local is initialised ONCE and keeps its value between calls.
P5	A	`5`	strlen stops at the first null character '\0'.
P6	D	`4`	3 characters plus the terminating '\0' = 4 bytes. strlen(s) would be 3.
P7	B	`0`	An unsigned char holds 0–255 (8 bits), so 255 + 1 wraps around to 0.
P8	A	`0`	Casting to int truncates toward zero: 7 + (−7) = 0.
P9	C	`10`	The inner i is a different variable that only exists inside the braces (it shadows the outer one).
P10	B	`21`	4 + 7 + 1 + 9 = 21, added from the last element back to the first.
P11	C	`30x1020`	Evaluation goes left to right: 10 + 20 = 30 (numbers), then everything after the "x" is string joining.

Q	Ans	Correct	Why
P12	D	`abc`	concat returns a NEW string, which is ignored here, and s itself never changes.
P13	C	`dcba`	StringBuilder changes in place: "abcd" is reversed to "dcba".
P14	B	`98 b`	char + int gives an int (97 + 1 = 98). Casting it back to char gives 'b'.

HANDS-ON DSA

✓2 Practice Set 2 · Data Structures & Algorithms

Each answer was **computed by actually running** the stack, queue, BST, heap, sorting pass, binary search or hash table in code.

D1 Stack **Starting with an empty stack: push 10, push 20, pop, push 30, push 40, pop, pop. In what order are the values popped?**

A 40 30 20
B 20 40 30
C 10 20 30
D 20 30 40

D2 Postfix **Evaluate the postfix expression `6 2 3 + - 3 8 2 / + *`**

A 49
B 1
C 7
D -7

D3 Infix → postfix **Convert `A + B * C - D` to postfix.**

A + A * B C - D
B A B C + * D -
C A B C * + D -
D A B + C * D -

D4 BST traversal **Insert 50, 30, 70, 20, 40, 60, 80 into an empty BST. What is the post-order traversal?**

A 20 40 60 80 30 70 50
B 50 30 20 40 70 60 80
C 20 30 40 50 60 70 80
D 20 40 30 60 80 70 50

D5 BST pre-order **For the same BST (50, 30, 70, 20, 40, 60, 80), what is the pre-order traversal?**

A 50 70 80 60 30 40 20
B 20 30 40 50 60 70 80
C 50 30 20 40 70 60 80
D 20 40 30 60 80 70 50

D6 Skewed BST **Inserting 10, 20, 30, 40 (already sorted) into an empty BST produces how many levels?**

A 4
B 1
C 3
D 2

D7 Min-heap **Insert 5, 3, 8, 1 one by one into an empty array-based min-heap. What is the final array?**

A [1, 3, 8, 5]
B [1, 5, 8, 3]
C [1, 3, 5, 8]
D [5, 3, 8, 1]

D8 Bubble sort **What is `[5, 1, 4, 2, 8]` after the FIRST pass of bubble sort (ascending)?**

A [1, 4, 2, 5, 8]
B [1, 5, 4, 2, 8]
C [1, 2, 4, 5, 8]
D [5, 1, 4, 2, 8]

D9 Selection sort — What is `[64, 25, 12, 22, 11]` after the FIRST pass of selection sort?

A [25, 12, 22, 11, 64]
B [11, 25, 12, 22, 64]
C [11, 12, 22, 25, 64]
D [11, 64, 25, 12, 22]

D10 Insertion sort — What is `[5, 3, 4, 1, 2]` after TWO passes of insertion sort?

A [1, 2, 3, 4, 5]
B [3, 5, 4, 1, 2]
C [3, 4, 1, 5, 2]
D [3, 4, 5, 1, 2]

D11 Binary search — How many comparisons does binary search need to find 23 in `[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]`? (mid = (lo + hi) / 2)

A 6
B 4
C 1
D 3

D12 Hashing — Table size 7, h(k) = k mod 7, collisions resolved by linear probing. Insert 10, 17, 24. At which index does 24 end up?

A 5
B 6
C 4
D 3

D13 1D address — int A[50] has base address 1000, and an int is 4 bytes. What is the address of A[10]?

A 1010
B 1044
C 1036
D 1040

D14 2D address — int A[3][4] is stored row-major at base 2000, with 4-byte ints. What is the address of A[2][1]?

A 2036
B 2032
C 2024
D 2009

D15 Circular queue — In a circular queue with indices 0–4 (size 5), rear = 4. After one enqueue, rear = ?

A 5
B 1
C 0
D 4

D16 Binary tree capacity — What is the maximum number of nodes in a binary tree of height 3 (root at height 0)?

A 16
B 7
C 15
D 8

Set 2 · Answer key

Q	Ans	Correct	Why
D1	B	`20 40 30`	LIFO: the first pop gives 20. Then 30 and 40 are pushed, so the next pops give 40 and then 30. 10 stays on the stack.
D2	C	`7`	2 + 3 = 5, then 6 − 5 = 1. 8 / 2 = 4, then 3 + 4 = 7. Finally 1 × 7 = 7.
D3	C	`A B C * + D -`	* binds tighter, so B * C is grouped first. Then +, then − (both left to right).

Q	Ans	Correct	Why
D4	D	`20 40 30 60 80 70 50`	Post-order is Left, Right, Root: the root (50) comes last. In-order would give the sorted list.
D5	C	`50 30 20 40 70 60 80`	Pre-order is Root, Left, Right.
D6	A	`4`	Sorted input makes a right-skewed chain, one node per level. Search becomes O(n), which is why balanced trees (AVL, Red-Black) exist.
D7	A	`[1, 3, 8, 5]`	Each insert goes at the end and sifts up while it's smaller than its parent. 1 rises to the root, and 5 moves down to index 3.
D8	A	`[1, 4, 2, 5, 8]`	Adjacent pairs are swapped when out of order, so the largest value (8) ends in its final place. 5 bubbles past 1, 4 and 2.
D9	B	`[11, 25, 12, 22, 64]`	The minimum (11) is found and swapped into index 0, and 64 moves to 11's old place.
D10	D	`[3, 4, 5, 1, 2]`	Pass 1 inserts 3, giving [3, 5, ...]. Pass 2 inserts 4, giving [3, 4, 5, ...]. The left part is always sorted.
D11	D	`3`	mid = 4 (16 < 23), then mid = 7 (56 > 23), then mid = 5, which is 23: found in 3 comparisons.
D12	A	`5`	10 → 3. 17 → 3 is taken, so 4. 24 → 3 and 4 are taken, so 5.
D13	D	`1040`	Address = base + index × size = 1000 + 10 × 4.
D14	A	`2036`	Row-major: base + (i × columns + j) × size = 2000 + (2 × 4 + 1) × 4.
D15	C	`0`	rear = (rear + 1) mod size. It wraps back to the start, which is the whole point of a circular queue.
D16	C	`15`	2^(h+1) − 1 = 15. Level i holds at most 2^i nodes: 1 + 2 + 4 + 8.

DBMS & CONCEPTS

✓3 Practice Set 3 · SQL & Theory

Q1–Q11 run on the **sample Employee and Department tables**. Every answer comes from executing the query on a real SQLite database. T1–T15 test concepts across the whole section.

Q1 SQL **Using the sample tables, what does this return?**

```
SELECT COUNT(*) FROM Employee WHERE salary > 45000;
```

A 4
B 3
C 2
D 6

Q2 SQL **Using the sample tables, what does this return?**

```
SELECT COUNT(dept_id) FROM Employee;
```

A 6
B 3
C 4
D 5

Q3 SQL **Using the sample tables, what does this return?**

```
SELECT COUNT(DISTINCT dept_id) FROM Employee;
```

A 4
B 6
C 3
D 5

Q4 SQL **Using the sample tables, what does this return?**

```
SELECT COUNT(*) FROM (SELECT dept_id FROM Employee GROUP BY dept_id HAVING AVG(salary) > 50000);
```

A 2
B 0
C 1
D 3

Q5 SQL **Using the sample tables, what does this return?**

```
SELECT COUNT(*) FROM Employee e JOIN Department d ON e.dept_id = d.dept_id;
```

A 24
B 4
C 5
D 6

Q6 SQL **Using the sample tables, what does this return?**

```
SELECT COUNT(*) FROM Department d LEFT JOIN Employee e ON e.dept_id = d.dept_id;
```

A 10
B 4
C 5
D 6

Q7 SQL **Using the sample tables, what does this return?**

```
SELECT d.dname FROM Department d LEFT JOIN Employee e ON e.dept_id = d.dept_id WHERE e.id IS NULL;
```

A Sales
B HR
C Finance
D IT

Q8 SQL **Using the sample tables, what does this return?**

```
SELECT e.name FROM Employee e JOIN Employee m ON e.manager_id = m.id WHERE m.name = 'Ravi' ORDER BY e.name;
```

A Meena, Arjun
B Asha
C Ravi
D John, Priya

Q9 SQL **Using the sample tables, what does this return?**

```
SELECT name FROM Employee WHERE salary > (SELECT AVG(salary) FROM Employee) ORDER BY name;
```

A Ravi, Meena
B Asha, John, Priya
C Meena
D Arjun, Meena, Ravi

Q10 SQL **Using the sample tables, what does this return?**

```
SELECT MAX(salary) FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee);
```

A 51000
B 55000
C 45000
D 62000

Q11 SQL **Using the sample tables, what does this return?**

```
SELECT COUNT(*) FROM (SELECT dept_id FROM Employee UNION SELECT dept_id FROM
Department);
```

A 10
B 5
C 6
D 4

T1 Concept **A program that translates the WHOLE source code into machine code before it runs is a:**

A Compiler
B Loader
C Assembler
D Interpreter

T2 Concept **Java source code is compiled into:**

A Assembly language
B HTML
C Native machine code only
D Bytecode, run by the JVM

T3 Concept **The range of a signed 8-bit integer is:**

A 0 to 255
B −127 to 127
C −128 to 127
D −256 to 255

T4 Concept **Where are local variables and function call frames stored?**

A Registers only
B Heap
C Stack
D Code segment

T5 Concept **Memory allocated with malloc/new that is never freed causes a:**

A Memory leak
B Stack overflow
C Dangling pointer
D Segmentation fault every time

T6 Concept **Which structure is NON-linear?**

A Linked list
B Stack
C Tree
D Queue

T7 Concept **Which traversal of a BST visits the keys in sorted (ascending) order?**

A Level-order
B In-order
C Post-order
D Pre-order

T8 Concept **The time complexity of `for (i = n; i > 0; i = i / 2)` is:**

A O(n)
B O(log n)
C O(n log n)
D O(1)

T9 Concept **The recurrence T(n) = 2T(n/2) + O(n) describes:**

A Linear search, O(n)
B Bubble sort, $O(n^2)$
C Binary search, O(log n)
D Merge sort, O(n log n)

T10 Concept **Quick sort's WORST case, for example on already-sorted input with a bad pivot, is:**

A $O(n^2)$
B O(log n)
C O(n)
D O(n log n)

T11 Concept **Which sort works well when the data is *nearly sorted*?**

A Insertion sort
B Selection sort
C Quick sort with the first element as pivot
D Heap sort

T12 Concept **A table is in 2NF but a non-key column depends on another non-key column. That violates:**

A BCNF only
B 2NF
C 3NF
D 1NF

T13 Concept **Which SQL clause filters GROUPS after aggregation?**

A ORDER BY
B GROUP BY
C WHERE
D HAVING

T14 Concept **'A committed transaction survives a power failure' is which ACID property?**

A Isolation
B Consistency
C Durability
D Atomicity

T15 Concept **Reading data written by another transaction that later rolls back is a:**

A Deadlock
B Non-repeatable read
C Dirty read
D Phantom read

Set 3 · Answer key

Q	Ans	Correct	Why
Q1	B	3	55000, 62000 and 51000. 45000 isn't strictly greater.
Q2	D	5	COUNT(column) skips NULLs. Arjun's dept_id is NULL. COUNT(*) would give 6.
Q3	C	3	The distinct non-NULL values are 10, 20 and 30.
Q4	A	2	GROUP BY puts all NULLs into ONE group of their own. The averages are 10 → 42500, 20 → 58500, 30 → 38000 and NULL (Arjun) → 51000, so two groups pass the HAVING filter.
Q5	C	5	An INNER JOIN keeps only matching rows: Arjun (NULL dept) and Finance (no employees) drop out.

Q	Ans	Correct	Why
Q6	**D**	`6`	Every department is kept. Finance has no match, so it appears once with NULLs: 2 + 2 + 1 + 1.
Q7	**C**	`Finance`	LEFT JOIN + IS NULL is the classic pattern for finding rows with NO match.
Q8	**D**	`John, Priya`	A self join: the table joined with itself to find each employee's manager.
Q9	**D**	`Arjun, Meena, Ravi`	The average salary is 291000 ÷ 6 = 48500. The subquery runs first.
Q10	**B**	`55000`	The classic second-highest salary query.
Q11	**B**	`5`	UNION removes duplicates: 10, 20, 30, 40 and NULL. UNION ALL would keep all 10 rows.
T1	**A**	`Compiler`	An interpreter translates and runs line by line. An assembler converts assembly language to machine code.
T2	**D**	`Bytecode, run by the JVM`	That's how Java is "write once, run anywhere". The JIT compiler then converts hot bytecode to native code.
T3	**C**	`−128 to 127`	-2^7 to $2^7 - 1$. In general, n bits give -2^{n-1} to $2^{n-1} - 1$.
T4	**C**	`Stack`	Dynamically allocated memory (malloc/new) lives on the heap.
T5	**A**	`Memory leak`	A dangling pointer is one that points to memory that has already been freed.
T6	**C**	`Tree`	Trees and graphs are non-linear. Arrays, linked lists, stacks and queues are linear.
T7	**B**	`In-order`	Left, Root, Right.
T8	**B**	`O(log n)`	i halves each time, so the loop runs about $\log_2 n$ times.
T9	**D**	`Merge sort, O(n log n)`	Binary search is T(n) = T(n/2) + O(1).
T10	**A**	`O(n²)`	Its average case is O(n log n). Random or median pivots avoid the worst case.
T11	**A**	`Insertion sort`	Insertion sort's best case is O(n).
T12	**C**	`3NF`	A transitive dependency breaks 3NF.
T13	**D**	`HAVING`	WHERE filters rows before grouping, and can't use aggregates.
T14	**C**	`Durability`	It's achieved through logs and writes to disk.
T15	**C**	`Dirty read`	Isolation levels such as Read Committed prevent dirty reads.

GET EXAM-READY

6-Day Plan & Cheat Sheet

Day	Plan (60–90 minutes)
Day 1	F1–F3: compiler vs interpreter, type sizes and ranges, casting, scope. Do P1–P10.
Day 2	F4–F5: call by value vs reference, the memory layout, pointers, array address formulas, string methods. Do P11–P14.
Day 3	F6–F7: stack and queue traces, infix/postfix, linked-list complexity, tree traversals (draw 3 trees), heaps, hashing.
Day 4	F8–F9: do one pass of every sort by hand, learn the sorting table and the Big-O patterns and recurrences. Do Set 2.
Day 5	DB1–DB4: keys, normal forms with examples, SQL families, execution order, joins on the sample tables, ACID. Do Set 3.
Day 6	Redo the questions you got wrong and read the cheat sheet.

Programming

- Compiler = whole program first · interpreter = line by line · Java → bytecode → JVM
- Java: int 4 bytes, char 2, long 8, double 8
- Signed n-bit: -2^{n-1} to $2^{n-1} - 1$ · overflow wraps
- Widening is automatic · narrowing needs a cast (truncates)
- static local: initialised once and keeps its value
- Stack = locals and calls · heap = malloc/new
- p + 1 moves one element · *(a + i) == a[i]
- Leak = never freed · dangling = points to freed memory
- Address = base + i × size · row-major (i·cols + j)
- strlen excludes \0 · sizeof includes it
- Java Strings are immutable · use StringBuilder
- 1 + 2 + "x" = "3x" · "x" + 1 + 2 = "x12"

DSA

- Stack LIFO (calls, undo, DFS) · queue FIFO (BFS)
- Circular rear = (rear + 1) mod n
- Linked list: head insert O(1), access O(n)
- In-order of a BST = sorted · pre = Root L R · post = L R Root
- Max nodes $2^{h+1} - 1$ · n nodes → n − 1 edges
- Heap children 2i + 1, 2i + 2 · parent (i − 1)/2
- Hash collisions: chaining / linear probing
- Binary search needs sorted data · O(log n)
- Merge always n log n · quick worst n^2 · insertion best n
- Stable: bubble, insertion, merge, counting
- Halving → log n · nested → n^2 · 2 branches → 2^n
- $2T(n/2) + n = n \log n$ · $T(n/2) + 1 = \log n$

DBMS

- Degree = columns · cardinality = rows
- PK = unique + not null · FK → another table's PK
- Candidate = minimal super key
- 1NF atomic · 2NF no partial · 3NF no transitive
- DDL: CREATE, ALTER, DROP, TRUNCATE
- DML: INSERT, UPDATE, DELETE · TCL: COMMIT, ROLLBACK
- DELETE (WHERE, rollback) · TRUNCATE (all rows) · DROP (table)
- Executed: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
- COUNT(*) counts rows · COUNT(col) skips NULLs
- INNER = matches · LEFT = all left rows · CROSS = m × n
- UNION removes duplicates · UNION ALL keeps them
- ACID · dirty / non-repeatable / phantom reads

Disclaimer: This is an independent preparation guide for informational purposes only, not affiliated with or endorsed by Accenture. C type sizes depend on the platform (64-bit Linux shown). All questions are original. Code answers were produced with gcc and OpenJDK 21, and SQL answers with SQLite. Standard SQL behaviour is assumed, and other database engines may differ in minor details such as NULL sort order.